

GoSendeet Security Audit

Audit date: 2026-04-30
Scope reviewed: frontend SPA, client-side auth/session model, exposed API contract, public dispatch flow, deployment/build config, and dependency posture inferable from this repository.

1. Vulnerability Summary

- * Critical: 0
- * High: 5
- * Medium: 2
- * Low: 2

Threat model:

- * Anonymous user: can hit public marketing pages, sign-in flows, password reset, tracking, and public dispatch links.
- * Authenticated user: can manipulate browser state, tamper with requests, and attempt horizontal or vertical privilege escalation.
- * Privileged admin/operator: can create API credentials, dispatch partners, and interact with high-value business operations.
- * Insider or compromised third party: can abuse support widgets, copied links, browser extensions, analytics, and CI/dev tooling.

Primary trust boundaries:

- * Browser storage vs server-issued session cookies
- * Public dispatch bearer links vs authenticated dashboard flows
- * Third-party scripts vs first-party authenticated UI
- * Frontend-controlled identifiers vs backend authorization checks

Sensitive assets:

- * Session cookies, reset tokens, dispatch access tokens
- * User identities and roles
- * Raw API client secrets
- * Booking/order/task metadata and customer delivery information
- * Any material encrypted or signed with the committed 'VITE_SECRET_KEY'

2. Detailed Findings

Title:

Client-side authorization state is trusted from 'sessionStorage'

Severity: High

Affected component:

Route guards, session bootstrap, user identity selection

Evidence:

'src/lib/authSession.ts:25-38', 'src/lib/AdminRoutes.tsx:7-25',
'src/lib/PrivateRoutes.tsx:7-25', 'src/lib/FranchiseRoutes.tsx:7-25',
'src/services/user.ts:5-16'

Description:

The SPA treats 'authSession', 'userId', and 'role' stored in 'sessionStorage' as the signal for whether a user is authenticated and which dashboard they may access. That state is writable by any JavaScript executing in the origin, by the user in devtools, and by malicious browser extensions. The frontend also uses the stored 'userId' to choose which profile to fetch.

Exploitation scenario:

1. An attacker gains script execution through XSS, a compromised third-party script, or a malicious extension.
2. The attacker sets 'sessionStorage.authSession=true', changes 'role' to 'admin', and swaps 'userId' to another target.
3. The SPA unlocks privileged routes and begins issuing sensitive requests for the attacker-chosen identity.
4. Any backend endpoint with weak authorization or missing object-level checks is immediately exposed.

Impact:

This does not by itself prove backend compromise, but it removes an important defense layer, makes latent BOLA/IDOR bugs much easier to exploit, and turns any XSS into an instant privilege-escalation primitive.

Recommended fix:

Treat browser storage as display cache only. Derive auth and role exclusively from a server-validated session or signed access token, fetch the current principal from '/auth/session' on load, and never let the browser choose the canonical 'userId' or role for privileged requests.

Title:

Third-party support JavaScript runs on every page without CSP hardening

Severity: High

Affected component:

Global 'ChatwootWidget', authenticated dashboards, deployment headers

Evidence:

'src/components/ChatwootWidget/index.tsx:3-21', 'src/App.tsx:35-92',
'index.html:1-55', 'vercel.json:1-4'

Description:

A third-party script is appended into 'document.head' for all routes, including authenticated and administrative screens. The app does not define a Content Security Policy, a strict 'Referrer-Policy', or any deployment-level security headers in the repo. If the support platform, its DNS, or a script dependency is compromised, the attacker gets same-origin script execution inside privileged sessions.

Exploitation scenario:

1. The Chatwoot asset origin or a transitive dependency is compromised.
2. The malicious script loads automatically for admins, franchise partners, and end users.
3. It reads 'sessionStorage', captures visible secrets, and performs authenticated API actions via the user's cookies.

4. It can silently create client credentials, copy dispatch links, or exfiltrate profile/order data.

Impact:

Full browser-session compromise across all roles, with especially severe impact for admin users because the widget executes inside the same trust domain as credential management and dispatch tooling.

Recommended fix:

Remove third-party scripts from privileged routes or isolate them in a sandboxed iframe on public-only pages. Add a strict CSP, 'Referrer-Policy', 'Permissions-Policy', and only allow explicitly pinned script origins. Prefer server-side support integrations over live script injection where possible.

Title:

Bearer secrets are transmitted in URLs for dispatch access and password reset

Severity: High

Affected component:

Public dispatch flow, password reset flow

Evidence:

'src/pages/dispatch/PublicDispatchPage.tsx:67-123',
'src/pages/admin/Orders/TaskManagement/modals/DispatchPreviewDialog.tsx:181-220',
'src/pages/auth/ResetPassword/index.tsx:16-19', 'src/services/auth.ts:102-116'

Description:

Public dispatch access is granted through '?access=...' URL parameters, and password reset consumes 'resetToken' from the query string. Query-string bearer tokens are routinely copied into browser history, screenshots, logs, chat systems, monitoring tools, and email clients. The dispatch token is removed only after client-side redemption; the reset token is never scrubbed from the URL in this flow.

Exploitation scenario:

1. An admin copies a dispatch link from the UI or a user opens a password-reset link on a monitored/shared device.
2. The full URL is exposed through history, clipboard sync, support transcripts, email forwarding, or endpoint telemetry.
3. A second party reuses the stolen bearer value before expiry or before the legitimate user finishes the flow.
4. The attacker accepts/declines/completes dispatch tasks or resets the account password.

Impact:

Unauthorized operational control of deliveries and potential full account takeover for password-reset victims.

Recommended fix:

Do not place bearer secrets in query strings. Use one-time POST redemption, URL fragments plus immediate 'history.replaceState', short expiries, replay detection, device/session binding, and step-up verification for sensitive flows.

Title:

A real-looking secret is committed in a 'VITE_' client environment variable

Severity: High

Affected component:

Secret management, build pipeline, any downstream crypto usage

Evidence:

`‘.env:1-5’`

Description:

The repository contains `‘VITE_SECRET_KEY’`, and Vite exposes all `‘VITE_’` variables to browser code at build time. That means this value must be treated as public. Even if it is currently unused by the SPA, the naming strongly suggests it was intended to be secret, which indicates a broken secret-handling process and creates a serious risk if the same key is reused elsewhere.

Exploitation scenario:

1. An attacker inspects the repository or built frontend bundle.
2. The attacker extracts `‘VITE_SECRET_KEY’`.
3. If the same key is reused for encryption, signing, token generation, or backend configuration, the attacker forges or decrypts protected material.
4. Even if not reused, the leak demonstrates unsafe developer behavior that often correlates with additional hidden secret reuse.

Impact:

Potential cryptographic bypass or token forgery if reused, plus guaranteed loss of confidentiality for the committed value itself.

Recommended fix:

Rotate the exposed key immediately, remove it from git history if feasible, keep secrets server-side only, and reserve `‘VITE_’` variables for values safe to disclose publicly.

Title:

Raw API client secrets are issued directly to the browser

Severity: High

Affected component:

Admin credential management

Evidence:

`‘src/services/clientCredentials.ts:25-28’`,
`‘src/services/clientCredentials.ts:113-148’`,
`‘src/pages/admin/Credentials/index.tsx:130-140’`

Description:

The admin UI requests new API credentials and receives the raw secret into JavaScript memory, where it is stored in React state and shown to the operator. In a hardened environment this is still a high-risk pattern, because any XSS, third-party script compromise, shoulder-surfing, browser malware, or extension compromise turns into long-lived API credential theft.

Exploitation scenario:

1. An attacker compromises an admin browser session through a malicious script or extension.
2. The attacker waits for the operator to create a new client credential.
3. The raw secret is captured from the response or from in-memory UI state.

4. The attacker uses the secret outside the browser, persisting access even after the admin logs out.

Impact:

Long-lived API compromise that survives normal session cleanup and may allow direct backend access from outside the web app.

Recommended fix:

Move credential issuance into an isolated administrative flow with step-up auth, strict origin isolation, and audit logging. Prefer showing the secret once from a dedicated backend page or downloadable artifact instead of keeping it in the SPA's general-purpose runtime.

Title:

Cookie-authenticated mutations show no visible CSRF protection

Severity: Medium

Affected component:

Authenticated API client and state-changing endpoints

Evidence:

'src/services/axios.ts:7-19', 'src/services/auth.ts:123-149',
'src/services/clientCredentials.ts:140-168', 'src/services/companies.ts:4-78'

Description:

The frontend uses 'withCredentials: true' for both auth and application API clients, but the repo shows no CSRF token acquisition, no 'X-CSRF-*' header injection, and no explicit origin-checking mechanism. If the backend allows cross-site credentialed requests through permissive cookie settings or a future CORS mistake, many state-changing endpoints become CSRF candidates.

Exploitation scenario:

1. A logged-in victim visits an attacker-controlled site.
2. The attacker coerces the browser into sending authenticated requests to the API using the victim's cookies.
3. Because the client contract contains no anti-CSRF token, the backend must rely entirely on cookie attributes and server-side origin enforcement.
4. Any server-side gap results in unauthorized state changes such as account actions, company updates, or credential revocation/creation.

Impact:

Forced actions in a victim session, particularly dangerous for admin and franchise roles.

Recommended fix:

Add a synchronizer or double-submit CSRF token, verify 'Origin'/'Referer' on sensitive endpoints, and enforce 'SameSite=Lax' or 'Strict' unless a narrowly justified exception exists.

Title:

The two-step sign-in flow likely leaks account existence

Severity: Medium

Affected component:

Authentication and verification flows

Evidence:

'src/services/auth.ts:43-49', 'src/pages/auth/Signin/index.tsx:35-52',
'src/pages/auth/VerifyEmail/index.tsx:77-100'

Description:

Sign-in first calls '/auth' with only an email address and forwards backend error messages directly to the user. That pattern commonly reveals whether an email is registered, unverified, or blocked before a password is ever requested. The verification resend flow also returns user-facing success/error feedback tied to a supplied email address.

Exploitation scenario:

1. An attacker automates requests against the email-validation entry point.
2. They classify responses and timing differences for large email lists.
3. Confirmed accounts are targeted for password spraying, phishing, or social engineering.
4. Unverified accounts can be singled out for onboarding fraud or helpdesk abuse.

Impact:

User enumeration that materially improves credential-stuffing efficiency and targeted attacks.

Recommended fix:

Return uniform responses for unknown and known emails, rate-limit aggressively, add bot detection, and avoid surfacing backend-specific auth state during pre-authentication.

Title:

Verification resend throttling is enforced only in 'localStorage'

Severity: Low

Affected component:

Email verification resend UX

Evidence:

'src/pages/auth/VerifyEmail/index.tsx:14-18',
'src/pages/auth/VerifyEmail/index.tsx:41-73',
'src/pages/auth/VerifyEmail/index.tsx:77-100'

Description:

The resend cooldown is tracked in 'localStorage', which an attacker can clear or bypass simply by sending requests directly to the API. This is acceptable as a usability feature, but it provides no real abuse protection.

Exploitation scenario:

1. An attacker clears browser storage or skips the UI entirely.
2. They send repeated resend requests to the backend.
3. If server-side rate limiting is missing or weak, the victim receives verification email spam and the endpoint becomes a low-cost abuse primitive.

Impact:

Spam, support burden, and additional user-enumeration signal.

Recommended fix:

Keep the client-side timer for UX only, but enforce resend quotas and anomaly detection on the backend by account, IP, and device fingerprint.

Title:

The repo pins versions with published security advisories affecting local development and dependency hygiene

Severity: Low

Affected component:

Dependency and development security posture

Evidence:

'package.json:28', 'package.json:77-78'

Description:

The repo uses 'axios@^1.9.0' and 'vite@^6.2.0'. GitHub's advisory database shows published advisories affecting older Axios 1.x branches and Vite 6.0.0-6.4.1. The Vite issue is dev-server-specific, but it still matters if developers expose preview/dev instances or reuse this stack in CI.

Exploitation scenario:

1. A developer starts a vulnerable Vite dev server with network exposure.
2. An attacker connects to the dev WebSocket and reads arbitrary local files under the published Vite advisory conditions.
3. Separately, unresolved Axios advisory debt increases the chance that a future unsafe usage or transitive gadget chain becomes exploitable.

Impact:

Developer workstation or CI exposure rather than direct production compromise, plus weaker supply-chain hygiene.

Recommended fix:

Upgrade Vite to at least '6.4.2' or newer maintained versions and review Axios against current patched releases. Re-run dependency review regularly and avoid exposing dev servers beyond localhost.

Reference:

Vite advisory GHSA-p9ff-h696-f583 (GitHub Advisory Database, published 2026-04-06);
Axios advisories GHSA-fvcv-3m26-pcq and GHSA-jr5f-v2jv-69x6.

3. Attack Chains

Chain:

Support-script compromise to long-lived API takeover

1. Compromise the globally loaded Chatwoot script or any allowed script source.
2. Read 'sessionStorage' and silently operate inside an admin session.
3. Create a new API credential and capture 'rawSecret'.
4. Persist access outside the browser via the stolen API secret.

Chain:

Query-string leak to dispatch hijack

1. An admin copies a dispatch link containing '?access=...'.
2. The link is forwarded into chat, email, logs, or clipboard history.
3. An attacker redeems the bearer token before or alongside the intended recipient.
4. The attacker accepts, terminates, or completes dispatch tasks and injects false delivery state.

Chain:

XSS or extension abuse to privilege escalation

1. Gain script execution in the SPA.
2. Overwrite 'sessionStorage.role' and 'sessionStorage.userId'.
3. Let the app unlock admin or franchise routes and issue attacker-chosen requests.
4. Combine with any backend authorization gap to read or modify other users' records.

4. Secure Design Recommendations

Architectural improvements:

- * Move to a server-authoritative identity model. The browser should never be the source of truth for role or principal selection.
- * Separate public, authenticated, and highly privileged surfaces by origin or at least by CSP policy. Customer support widgets should never share the exact execution context of admin credential management.
- * Replace bearer URLs for sensitive operations with one-time POST redemption flows, short lifetimes, replay protection, and contextual binding.
- * Isolate API credential issuance behind step-up authentication, stronger auditing, and a dedicated privileged workflow.

Safer patterns and best practices:

- * Add strict response headers: CSP, 'Referrer-Policy', 'X-Frame-Options' or 'frame-ancestors', 'Permissions-Policy', and HSTS at the deployment layer.
- * Remove secrets from client env files and rotate any key that has ever been exposed to the frontend or committed to git.
- * Enforce backend CSRF protection, rate limiting, and object-level authorization on every ID-bearing endpoint, regardless of what the UI does.
- * Treat the current frontend as an attacker-controlled client when designing API authorization checks.
- * Upgrade Vite and review Axios against current advisories as part of dependency hygiene.